

1. Embracing Event-Driven Architectures

Hugo Filipe Oliveira Rocha¹

(1) Ermesinde, Portugal

This chapter covers:

- How monoliths can hinder business growth and their main limitations
- Understanding microservices, their advantages, and how they relate to event-driven architectures
- Recognizing the potential in adopting event-driven microservices and how an event-driven architecture works
- How to identify the need for your business to move to an event-driven architecture
- Knowing the challenges in adopting a microservice event-driven architecture

You might have struggled, the same way as I did, with the limitations of monoliths. It starts with a small team delivering value in a fast iterating working mode in a single application. The business succeeds, and the use cases grow and get more diversified. More developers join to keep up with demand. High-priority features and urgent fixes blur the lines of single responsibility and slowly turn the practices of clean code into remnants of the past. The team outgrows the application, and we soon struggle with merge requests with hundreds of conflicts and complex lockstep releases. The first time I migrated a monolith to a distributed microservice architecture, it felt like a glimmer of hope in the inhospitable big ball of mud¹ we so often and easily fall into.

It often starts smoothly: the first microservices stand as living proof of how time to market and team's autonomy can be software engineering pillars even in large teams working with complex architectures. The in-

conspicuous detail is the fundamentally different hell we are walking into that most people are oblivious to. While working in a global eCommerce platform, I recall a situation where one of the teams had to migrate the product's stock information to a new service. The operation was simple: the new service would fetch the stock information from the service holding the inventory data, perform some straightforward operations, and save the data internally. One of the operations the service had to do was fetching the location of the warehouse storing the stock, which involved requesting data from the location service. The inventory service, a crucial component in the platform, had plenty of resources, and the added throughput of the operation was accounted for. However, no one considered the impact in the location service, a component with apparently limited importance and relevance in the platform. The added throughput of the operation in the location service caused the service to crash due to high load. What we soon noticed was that the apparently minor and peripheral location service was being used in the order fulfillment flow and other vital parts of the platform. The apparent harmless data migration of a new service ended up stopping the main order fulfillment flow. To me, it felt like the gilding on the microservice architecture cracked to reveal the imperfections underneath.

When an apparently innocuous change ends up affecting a completely unrelated functionality, a distributed microservice architecture doesn't seem all that different from a monolith. In fact, we might get the worst of both worlds and end up with a distributed monolith instead. This consequence is one of the many challenges we need to address when building a distributed microservice architecture. One of the most sustainable ways to address those challenges is through an event-driven architecture.

In a microservice event-driven architecture, several decoupled services react to each other and gracefully choreograph to accomplish a business goal. They provide an immutable sequence of events that enable you to understand the purpose and the evolution of the data and interpret their business processes. The hauntingly beautiful potential of event-driven architectures is the ability to provide streaming of data with history and meaning, enabling each consumer to take the vision that is more fitted to the consumer's context and build value on top of it.

Although event-driven architectures aren't new and were very popular with ESB (enterprise service bus, detailed in Section [1.3](#)), the ability to provide high volumes of messages to distributed components sustainably and in real time certainly is new. Without a doubt, for most use cases, the most valuable data is the most recent one, but being able to offer an accessible stream with the history of the data unlocks robust solutions to difficult problems. Take migrating data for example. Instead of complex syncing solutions that have to deal with data being changed while the migration is running, doing it with event streams becomes trivial and organic.

Microservices pushed the limits of traditional applications, but sometimes to unravel more difficult ones. Synchronous communications traditionally used with microservices (like REST) become problematic due to the inherent coupling. Although much safer than in a monolith, changes in services might still affect the underlying ecosystem inadvertently. The complex web of synchronous calls between services is hard to track and create a breeding ground for cascading failures. We also choose microservices due to the scalability capabilities, which can become hindered by the synchronous dependencies between services (further detailed in Section [1.4](#)). Although we still need deliberate approaches to these problems, event-driven architectures provide a higher level of decoupling and enable a genuinely evolutionary architecture from the ground up. New consumers can join without affecting existing services. Old services can be decommissioned without affecting their dependencies.

However, the asynchronous, distributed nature of event-driven architecture poses difficult challenges that we often take for granted in a monolith. Event-driven isn't also a silver bullet and shouldn't be used in every use case. In the end, complex businesses ultimately reflect their complexity in the technical solutions we develop. Our duty and our mission is to model that complexity in a way that enables the business to grow, either in scale, velocity, or new functionalities. An event-driven architecture proposes a way to do just that.

1.1 The Truth About Monoliths

This section will discuss the advantages and limits of monoliths and how they can constrain business growth. Often monoliths are treated as an anti-pattern, strongly associated with legacy applications. However, there's a lot of misconception about the meaning and usage of monoliths. Monoliths can be an adequate architectural decision depending on the situation, failing to acknowledge this, limits our options. Microservices and event-driven architectures aren't a one-size-fits-all solution; neither monoliths are a one-fits-none solution. We will further detail this in Subsections [1.1.1](#) and [1.1.2](#).

We can adopt many strategies to circumvent the monolith's limits, but some limitations reach the point where we need a more sustainable approach. The business growing pains start aching typically in monolithic applications and manifest themselves through a myriad of symptoms. When not tackled with a deliberate approach, this might limit the business's success and be a ball and chain hindering its advance. The common drawbacks of monoliths are detailed in Subsection [1.1.3](#).

However, deciding to move to a microservice event-driven architecture requires a deliberate decision and a strong reason. Event-driven microservices might be the solution to many of the monolith's limits, but they are also the source of more complex challenges. Moving to a distributed architecture requires thoughtful preparation and the right fertile conditions to thrive. We detail the movement to event-driven microservices in Subsection [1.1.4](#).

This section approaches these topics to give you an overall context. The detailed migration from a monolith to an event-driven architecture, with a use case and strategies to do this, is discussed in Chapter [2](#).

1.1.1 Anatomy of a Typical Monolith

Businesses start somewhere, sometimes with a piece of software. New functionality is added to that software; sometimes it is a success, sometimes it's not, and the business chooses a different route. If we are lucky enough, that business prospers and that piece of software grows. New functionality spawns inside the application, and old functionality, even when removed, leaves a footprint due to design choices or technical debt.

Without deliberate effort to decouple domains and a disciplined organization, it quickly grows into an application that's hard to maintain.

Patchwork Monoliths

Also known as big ball of mud, the most frequent type of monoliths are the ones that have been built during several years with no clear boundaries and logic entangled together. I like to call them patchwork because you usually see cores of domain logic concentrated in one place and then all over the place, like a patchwork quilt.

They are also the ones most troublesome to deal with and to maintain. The application is a single artifact and is deployed all at once. All of it is a single process running in a machine. Through the years, the domain logic becomes entwined and hard to read.

Figure [1-1](#) illustrates the example of an eCommerce platform with several domains. Without a deliberate effort to maintain each domain divided and decoupled, all the logic from the different domains becomes mashed together.

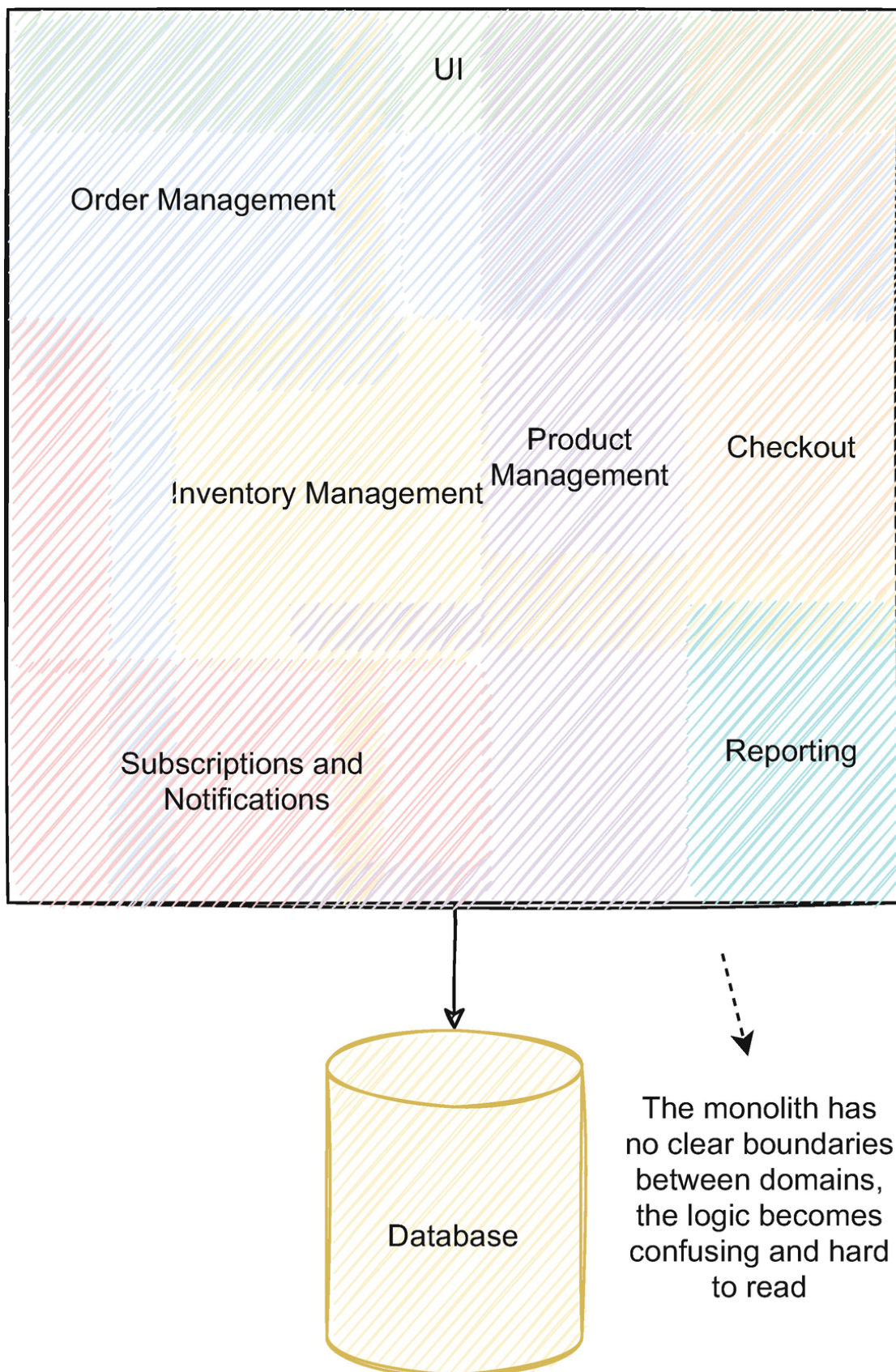


Figure 1-1 Example of an eCommerce patchwork monolith with no clear boundaries between each domain

This kind of monolith is the one that is more susceptible to the drawbacks explored in Subsection [1.1.3](#).

Modular Monoliths

Modular monoliths are divided into modules with clear boundaries; the code is decoupled and able to evolve independently. Modular monoliths benefit from all the advantages monoliths have (and yes, they do have some, detailed in Subsection [1.1.2](#)) and don't suffer from the complex challenges distributed architectures have.

Monoliths usually have a negative connection because most of them are patchwork. Still, a well-structured modular monolith can be an architectural option as valid as a microservice, SOA (service-oriented architecture), or event-driven architecture. Figure [1-2](#) shows the same example as before, but all the intertwined domains are organized in clear boundaries.

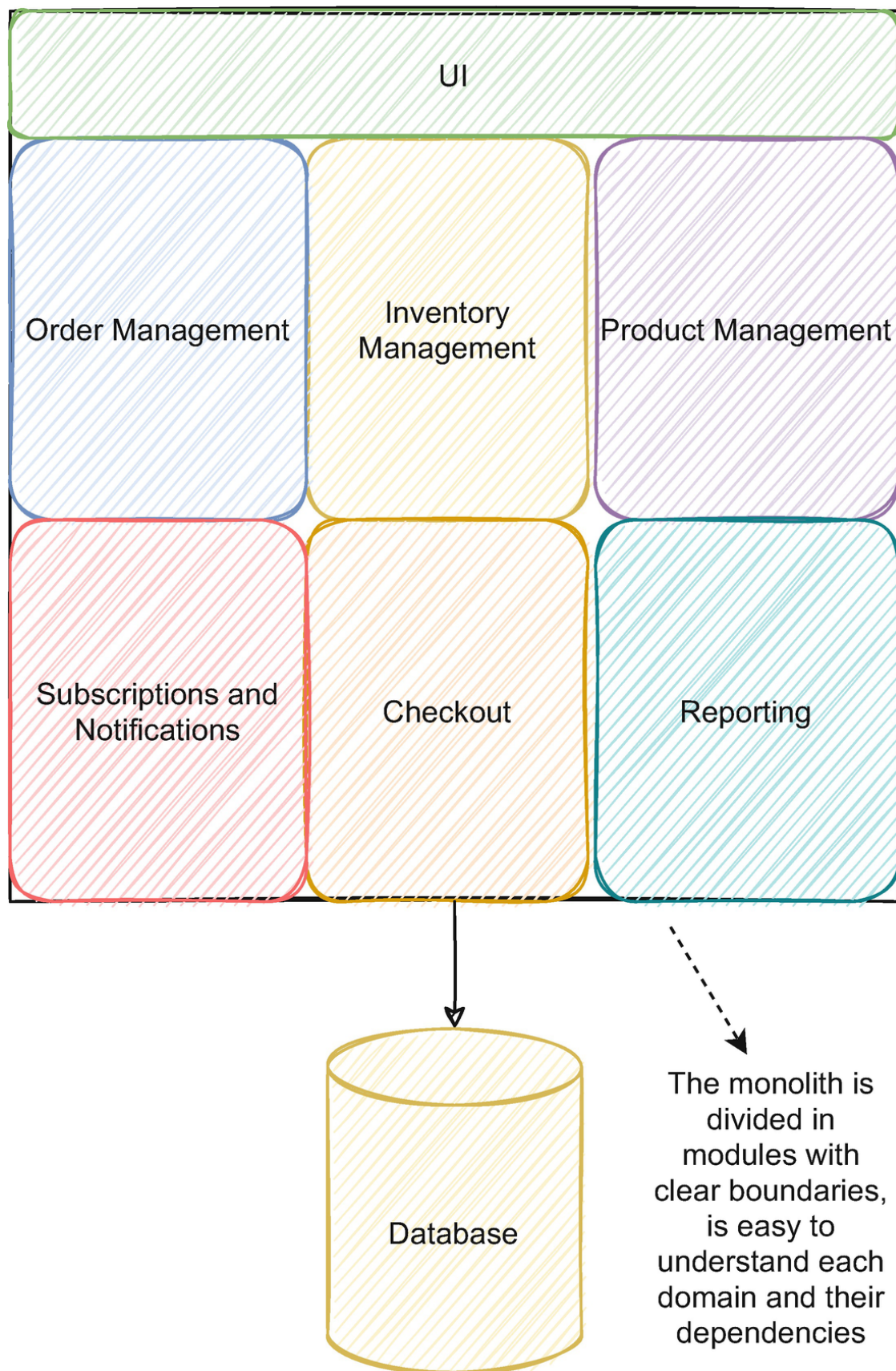


Figure 1-2 The same eCommerce platform with a modular approach; each domain has clear boundaries

The dependencies between each module are contained, organized, and visible. The modules should be able to evolve without affecting the other modules; even when we need to remove one of the modules, it is

possible because the dependencies are explicit and the domain is contained.

The common pitfall is how easy it is to violate that principle; the dependencies need a deliberate strategy to remain decoupled. That can be done with APIs for each boundary or automate tests for explicit dependency validation (this article² details how Root did this). It is still a single application that is deployed together; even if you change just one module, it often requires the deployment of all modules.

1.1.2 What They Don't Tell You About Monoliths; It's Not All Bad

Although monoliths have a terrible reputation, there are advantages to maintaining a single application with one codebase. We often miss those advantages due to being overwhelmed with their limits. There are fundamental, well-known developing principles that we often use on traditional single-process applications that become deficient when using a distributed system and require a different approach. A distributed system is more complex and doesn't have some of the characteristics we always took for granted in monoliths. This section will detail the main advantages of monoliths and how they can ease the delivery of features.

Business Flow Is Visible

With a single codebase, we can inspect the end-to-end flow quickly. We can effortlessly find any feature we want since all functionality is in one repository. It is also easier to see the impacts of new developments since we can see the dependencies of that flow. The business flow can become hard to read with event-driven architectures due to the asynchronous interaction between several services. To understand the business process, we need to understand the flow of events between each service. It is harder to understand the bigger picture, and the bigger picture is always important when adding new features, either to account for possible impacts or to understand if the development is the correct one for the feature. In a monolith, it is possible (although sometimes laborious) to understand how they fit in the overall flow.

No Network Overhead and Limited External Dependencies

All the different modules call each other directly inside the application. There are no remote calls through the network through external APIs or event brokers. This characteristic can enjoy a performance boost (although constrained to the scaling limits) since there is no network overhead. It also doesn't need to deal with API or event versioning and backward compatibility; if there is the need to do a breaking change, we can do it in a single release; all dependencies are inside the application. This characteristic simplifies the development and the release of features that need more profound changes.

We should always avoid a breaking change (as we will detail in Chapter 8), but sometimes they are unavoidable. In an event-driven architecture, we would need to support two versions of an event temporarily, or two versions of an API, to allow the consumers of that event to adapt to the new one. These changes are complex since they possibly affect several services. There is a need for coordination between different teams to align the shift to the new event. There is a considerable development overhead due to the development of the two events' publication and consumption and then removing the old one. In a monolith, this is straightforward; we just change that one application and release it.

Local Validation

In a monolith, it is possible to locally run the whole environment (although I saw applications that took several minutes just to build and additional several minutes just to start up locally). Often we can add and validate a feature by running a single application. However, on an event-driven architecture, we will be adding the feature to one of the several services in the workflow. It's often hard to run the whole workflow locally due to the complexity of the flow and the high number of services it requires, each with their specific configurations and unique dependencies (sometimes even in different languages and environments). Most of the services we might not even know that well since they belong to other teams.

Code Reuse

Since all the code is right there inside the application, it is easy to reuse and build on top of existing functionality. In an event-driven architecture, every service is isolated with their codebase, many of the services will need similar functionality, and obviously, it's not possible to share code. Creating custom frameworks to use across services can be useful, but it isn't as straightforward as using the code inside an application like a monolith and introduces coupling which might be hard to manage later on. Frameworks can also be difficult to debug and can be a nuisance if we need to update all services with a new version. Usually microservices prefer to duplicate code rather than reuse to avoid coupling. Reusing code also makes the code less useful, since it needs to deal with every possible use case. Frameworks are still a valuable asset on common infrastructural functionality like logging.

Monitoring and Troubleshooting

Monitoring a monolith is straightforward since it is only one application. There is a reduced number of machines to monitor, and the logs are trivial to fetch. Troubleshooting an issue is also easier since the scope is reduced to one application. In an event-driven architecture, there are several instances of dozens or hundreds of microservices. This requires a whole different strategy for monitoring and pinpointing incidents. There is a need for a pre-existing infrastructure to manage all those microservices' signals and metrics. Understanding an incident can also be a detective's journey worthy of a short novel without the deliberate approach. On the other hand, a monolith is much more straightforward since the information is far more localized.

End-to-End Testing

End-to-end testing in a monolith is considerably simpler than an event-driven architecture, and I will dare to say, more than simpler, it is actually possible. Since it is a single application, we can have automated tests that validate the whole application's flow. Often end-to-end validations are contained in that application since they don't depend on external services (other than databases, caches, etc.) and can be managed as a single piece. On an event-driven architecture, the approach has to shift from a

single piece approach to not having end-to-end tests (or a significantly reduced amount) and having other processes to guarantee quality (we will further detail these processes in Chapter [10](#)).

Simpler Deployment Strategy

Since there is only one application, the deployment pipeline needs to account only for that application's needs. A microservice architecture has to support the deployment of several different services, possibly written in several other languages and with several different dependencies. The overhead of building that kind of pipeline can be lesser or greater depending on the environment's diversity. The deployment topology is often much simpler (and cheaper) with a monolith than with a microservice architecture.

Data Is Centralized

For better or for worse, the data is typically centralized in one or many accessible databases. If a new feature needs some kind of data, it can just go and fetch it from the database. It is simpler to develop, but it has dire consequences when the application reaches a large scale. Although an advantage on the implementation side, it is also one of the most common reasons to leave a monolithic architecture. One of the challenges in event-driven architectures is fetching and managing data dependencies from other services (further detailed in Chapter [8](#)).

Possible to Scale

We can scale monoliths by installing several instances of that application behind a load balancer (also known as cookie cutter³ scaling). The application does need to be ready to deal with concurrent requests. However, it is limited to the scaling of the application, not the database; when the issue is the database, vertical scaling is often the only option. On the other hand, event-driven architectures are built for horizontal scaling from the ground up and are much easier to scale.

Consistency

Monoliths often use relational OLTP (online transaction processing) databases, which have strong consistency guarantees. These types of databases on monolithic applications typically enjoy ACID (atomicity, consistency, isolation, and durability) guarantees, which provide the traditional consistency guarantees we are used to; for example, a change happens everywhere at the same time. On event-driven architectures, due to the asynchronous nature of events, the consistency is often eventual consistency, which can be challenging to deal with (we further detail approaches to deal with eventual consistency in Chapter [5](#)).

Concurrency

In monolithic applications, we can deal with concurrency with the traditional strategies to handle race conditions, like locks or using the database. However, there might be multiple instances of a service on separate machines in an event-driven system, not being possible to do an in-memory lock. Also, depending on the database technology the service uses, it might not support transactions. Dealing with concurrent events requires different approaches that we further detail in Chapter [6](#).

1.1.3 When Monoliths Become the Business Constrictor Knot

This subsection will discuss the usual problems we associate with monoliths and how they can limit business growth. Monoliths can be an adequate solution in several contexts; however, as companies outgrow them, their handicaps become an increasing concern, especially with patchwork type monoliths. Companies typically struggle the most when reaching a large scale, both in data, usage, and developers. When the business pulls one of these factors because it needs or was successful, the monolith's limits pull the other way around. Like a constrictor knot, the more you pull, the tighter it pulls back. Given enough time, it can bring the business to a halt. It is essential to have a clear reason or a specific problem to tackle when migrating to an event-driven microservice architecture; often it is one or a combination of the ones discussed in this subsection.

Coupling and Lack of Boundaries

The main issue with monoliths is, given enough time, they become unbearably complex and coupled. Since all functionality is inside a single application, it is relatively easy to compromise each domain's boundaries. With time, they fade and become entwined and hard to read. A single change might affect several parts of the system and has unpredictable impacts, for example, changing the subscription logic affects the login. It becomes a nightmare to maintain and change reliably. Since a microservice event-driven architecture relies on events, it is decoupled by design, being easier to maintain the boundaries between domains through time.

Team's Autonomy

As the development team grows, it gets increasingly harder to work on a single application. Even with clearly defined modules, the development still requires communication and alignment, the deployment of the application needs coordination between the different teams. Depending on the development workflow, feature merges also become problematic. Feature branches will often lead to large merges with a high number of conflicts. The overhead of communication, quality assurance, and coordination increases with the number of developers working in the application. We often talk about scaling the application resources, but monolithic applications often limit the scaling of the team.

Release Cycle

The release cycle of a single monolithic application is usually larger than a microservice. Even if a company adopts continuous delivery, the cycles will be inherently larger due to every release; no matter how small the change, the whole application must be validated and deployed. Validating the entire application at once requires a gigantic test suite that can take a considerable amount of time to run (assuming there's no manual validation that often there is). Small changes make the deployment more controllable and enable fast feedback, making it harder to achieve with larger release cycles.

Typically, businesses that rely on monoliths don't deploy that often (e.g., every day) due to the risk since we always deploy the whole applica-

tion. Not deploying that often accumulates features, and it is harder to have a continuous delivery mindset. That alone is a common argument to move away from a monolith.

Scaling

Although scaling is possible, as we mentioned in the previous section, it is limited to the application. It also implies the monolith has the mechanisms to deal with concurrency with several different instances. When they don't, they require extensive changes to be able to deal with concurrent requests.

Most of the time, monoliths also nurture a monolithic database, which is very hard to scale. When the data starts to have a very high volume, it becomes problematic, especially with queries that require a lot of joins. Vertically scaling (increasing the number of resources, e.g., memory, CPU, etc.) is always an option for both the database and the application, but it gets costly very fast. If a business grows to a stage that needs geo-distribution to provide low latency to every location, a monolithic database might limit its ability to achieve it. A microservice approach shines in this particular aspect since each microservice owns its own database (or at least should), paving the way for a more scalable distribution of the data. Event-driven microservices by design are very easy to scale since they handle events from a queue and largely benefit from the associated decoupling.

When we scale a monolithic application, we scale the whole application. Most of the time, only a single or a part of the modules need scaling. But since every module is bound together, the only option is to scale the whole application. By using a microservice approach, it is possible to scale only the part of the architecture that needs scaling and leave the remaining parts with minimal resources, optimizing costs and resources.

Outdated Technology Stack

Most of the time, monoliths are applications that have several years. Meanwhile, some of the technologies might have been discontinued. It goes beyond the usual technology hype of adopting something just be-

cause it's the new best thing. If the application uses technology that is no longer supported by the company that made it, it is a concern. Suppose we face a compatibility issue with new versions of the operating systems or choose to adopt a different way to deploy the application (if we choose to deploy a .net framework application in a Linux environment, for example). In that case, it will limit our options if the monolithic technology doesn't support it. It can be a deal-breaker for new functionalities and can seriously hamper our ability to evolve. Most of the time, changing the whole technology stack on a monolith requires a monumental effort with unimaginable impacts.

Technical debt and no longer needed functionalities can be challenging to remove due to the coupling between boundaries. Although we can achieve this with a modular monolith (Shopify did this with their tax engine⁴), most of the time, removing or replacing a functionality involves touching the whole application and can have unforeseen consequences. Although any service can have an outdated technology stack, event-driven services are highly decoupled; implementing an existing service in a new technology is greatly simplified.

Reliability

The risk of deploying a monolith is that any change can bring the whole application down. Even a change in a less critical part of the application can produce an issue (like a memory leak or unforeseen CPU usage) that can affect the whole application, and the whole application is every functionality.

Event-driven architectures might not improve reliability directly since by using several instances on several different machines, we are increasing the number and types of failures that might occur. A deliberate approach to solve these issues is crucial. However, we can deploy smaller parts of the system independently, which (hopefully) won't bring the whole application down.

1.1.4 Using Event-Driven Architectures to Move Away from a Monolith

In the last few subsections, we discussed the advantages and limits of monoliths. In this subsection, we will discuss the transition phase between a monolith and an event-driven architecture. We will slightly approach the considerations we need to make to move to this architecture and how we can start. These topics will be further detailed in Chapter [2](#) with a real use case.

It is important to notice that an event-driven architecture isn't the ideal solution for every use case. As we discussed in Subsection [1.1.2](#), monoliths have a set of properties that make development easier, and the limits discussed in Subsection [1.1.3](#) can also be managed at least until a point. Moving to an event-driven architecture considerably increases the complexity and has costs associated with it. For example, it is debatable if a startup should adopt an event-driven architecture from the beginning. A startup typically is still trying to understand which products work and which don't, needs a fast time to market, needs to be innovative, and might need to restructure a large part of the solution. All that with limited funding, an event-driven microservice architecture has associated costs due to the infrastructure (deployment, monitoring, etc.) that might be overkill in such an early phase. Also, we can apply it best with a considerable understanding of the domain and its boundaries. Changing the boundaries after they are in place can be costly and laborious.

It is also essential to understand why we want to move to an event-driven architecture. Often the potential of this architecture (detailed in Section [1.6](#)) and the hype generated around it can be enticing. Still, one or combined factors mentioned in Subsection [1.1.2](#) are much stronger than the hype. Perhaps we are already struggling with one of the monolith's limits, maybe we are already struggling in scaling the application or the database, maybe our team is getting too large, perhaps we want a tool that enables us to adopt an Agile and continuous delivery mindset with ease. We should always have a strong reason to do the shift, and it should be a top priority while migrating. After electing that reason, we should measure how we are doing against it and understand if we are making progress or not (this is further detailed in Chapter [2](#)).

To make a move is vital to know where to start, what domains exist, and how they interact with each other. Figure [1-3](#) illustrates the possible steps to start the migration

from a patchwork monolith to a modular monolith. If the monolith is a patchwork monolith and most of the logic is coupled together, an important first move could be to refactor that monolith into a modular monolith. It wouldn't be unprecedented if most of our issues with a monolith disappeared when refactored to a well-structured modular monolith. Sometimes it's a complete nightmare to do that shift in a monolith. Either way, the exercise of trying to do that is pivotal to understanding the domain and associated boundaries, even if we don't change to a modular monolith. By knowing the boundaries, we can reason which would be the most adequate domain to start migrating. By knowing their dependencies, we can plan how to address them both on the monolith and the new architecture. If the monolith is modular, then part of this work is already done, the boundaries are defined, and it's easier to understand its dependencies.

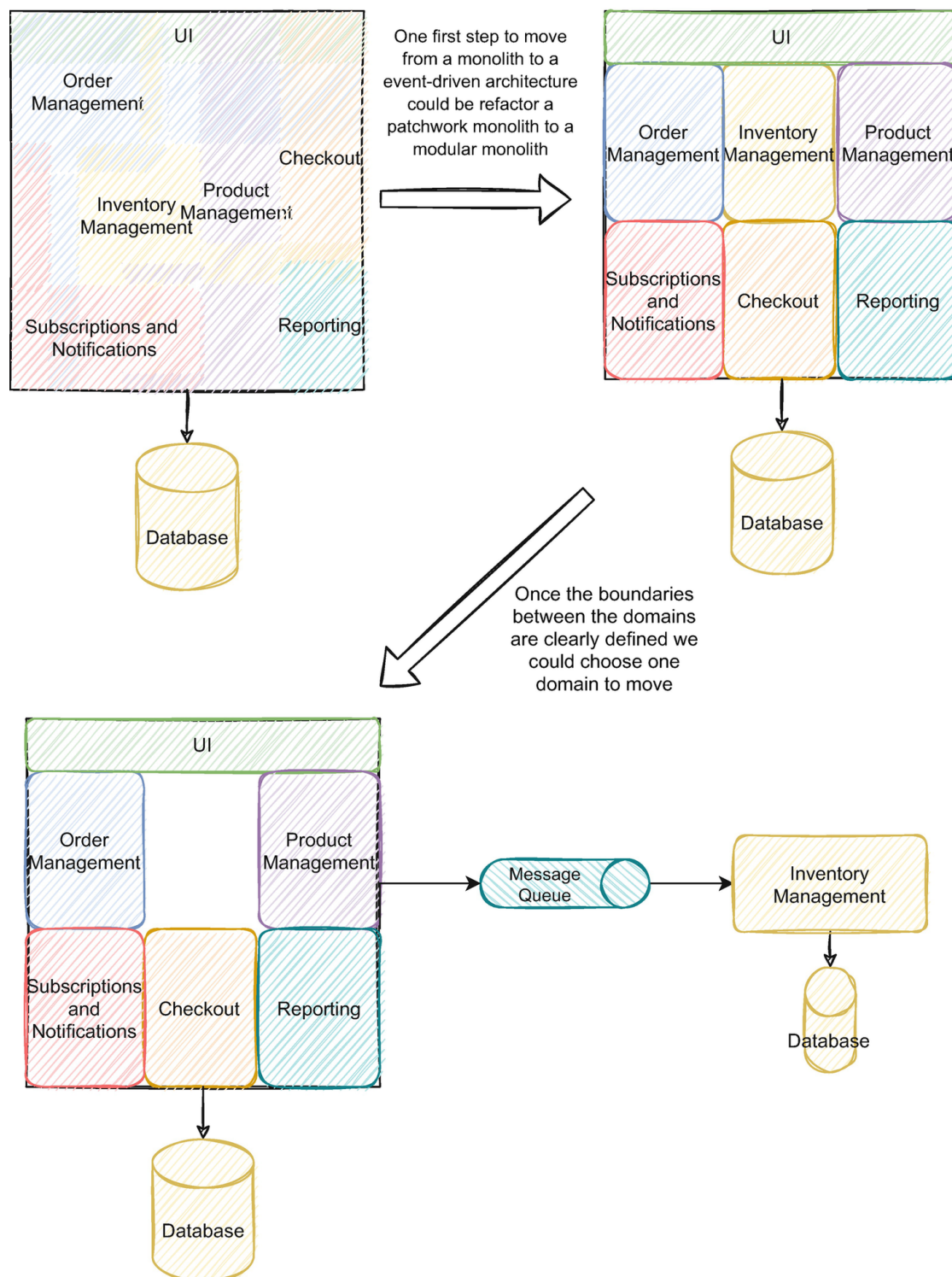


Figure 1-3 An example of the steps to start the shift from a monolith to an event-driven architecture

Choosing which domain to move to the new architecture isn't always easy; the decision has to weigh the risk, value, and viability. Risk is the business domain's criticality, value is the benefit we might obtain from migrating that domain, and viability is how practical and viable it is to migrate it.

Once we choose an appropriate domain, we can use an event-driven approach to decouple that domain from the monolithic application. In the

example of Figure [1-3](#), we moved the inventory management module to an independent service that reacts to changes occurring in the original monolith. However, this change introduces all the challenges distributed event-driven architecture has, which this book approaches.

The process of migrating a monolithic application to a microservice event-driven one is hardly a full one-shot migration, where we descope the monolith and create the whole new architecture from scratch. I have rarely seen this happen; most likely, you already have a monolith and might be starting to migrate it to a distributed architecture. Doing a greenfield event-driven microservice architecture also isn't the right option in most cases. It is essential to have a strong understanding of the domain and its boundaries before moving to this kind of architecture. It is hard to have an in-depth mastery of the system's domain without an existing application. In the beginning, those domains can change considerably; adopting an event-driven architecture early on can have substantial costs by reorganizing and redefining the domains.

Adopting a small increment migration enables us to understand what is working and what is not. It allows us to face the challenges of distributed event-driven architectures one step at a time while maintaining the business running. It gives us the space to fail, understand why we fail, and adapt accordingly at a low scale. It gives us room to adapt, learn, and act without having the world crashing down on us. Event-driven architectures also introduce a level of decoupling and reliability different from synchronous microservices. Event streams enable sustainable ways to access the data even from monolithic applications. We further discuss these strategies and the migration from a monolith to a microservice event-driven architecture in Chapter [2](#).

1.2 What Are Microservices and How Do They Relate to Event-Driven

In the last section, we discussed the monolith's characteristics and limits and how they can constrain business growth. We also approached the various considerations when starting a migration to a microservice event-

driven architecture. In this section, we will discuss what microservices are and how they fit into an event-driven architecture.

An event-driven microservice is a simple, specific purpose service that reacts to events to do its work. In an event-driven architecture, several of these services interact with each other to accomplish a higher-level process. Usually, the interaction is composed of a sequence of events between the different services.

An event-driven architecture accomplishes its business processes using the flow of messages through these loosely coupled services. Each service belongs to a given domain or bounded context and has a specific role and limited responsibilities inside that domain. Each domain has the responsibility to process the relevant data in that domain and communicate that change to other domains.

Using the example in Figure [1-4](#) of an eCommerce platform, when buying a product, the process of accomplishing that order would be the interaction of several boundaries, in this case, the order, inventory, and pricing boundary. The order management boundary processes the order, the inventory boundary manages the stock, and the pricing boundary manages the taxes and final prices for that country. Instead of being the synchronous flow in a single application is the choreography or orchestration of several individual services, each one with its role. Having single-purpose services enables organic bounds for each service and consequently for each boundary.

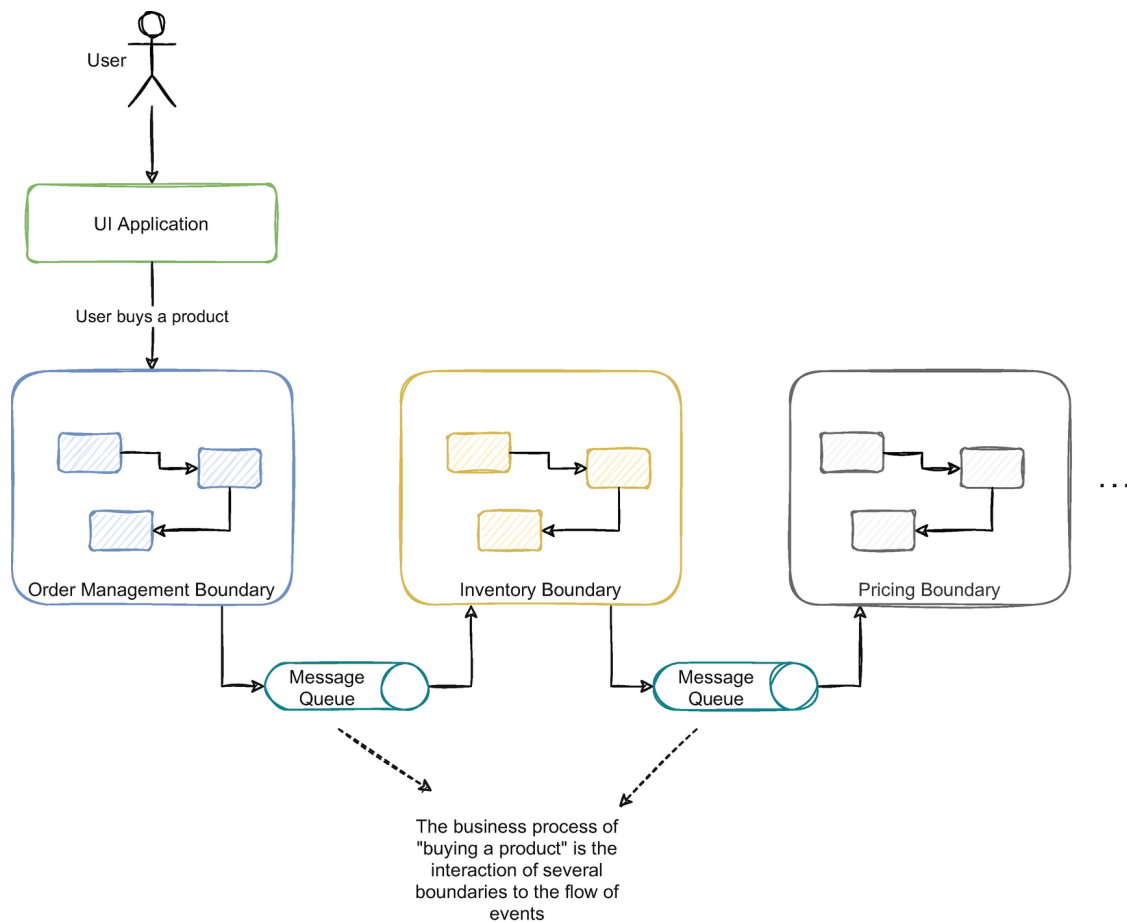


Figure 1-4 The user purchasing a product triggers a business process composed of the interaction between several boundaries

In event-driven microservices, each fully decoupled service reacts to each other through technology-agnostic messaging to complete a higher business process.

1.2.1 Deployment

We should also make event-driven microservices independently deployable; a common anti-pattern is the need to deploy several services together. When we need to do so, it rapidly becomes a nightmare to manage and guarantee reliable releases. Also, the deployment should be automated; the teams should be focused on delivering business value rather than laboriously pushing their release live through an intricate release process. The right tools need to be in place for an automated deployment with the required validations. Developers should rely on those deployment tools rather than release or DevOps teams.

1.2.2 Decoupled and Autonomously Developed

We should develop each service autonomously without dependencies from other services. They should also be fully decoupled, easily achieved due to the message brokers between the services. Event-driven enables the architecture to be highly evolvable (as mentioned in the book *Building Evolutionary Architectures*⁵); changes in a given service will hardly affect the whole architecture, and we can deploy the service independently. This characteristic also makes the architecture highly pluggable since new services can simply listen to the events already flowing through the architecture.

1.2.3 Data Ownership

Each service should own the data needed for its domain and expose it through a straightforward interface, typically messaging in an event-driven architecture (although an API is also an alternative). Owning the data is pivotal to achieve independent and autonomous developments. Otherwise, often a change in one service requires other services to change at the same time. Changes in the schema, manageable when in a single service, become a nightmare to release.

1.3 SOA, Microservice, and Event-Driven Architectures

SOA (service-oriented architecture) and microservice architecture often come up when talking about event-driven architecture. There's even some confusion about the differences between SOA and event-driven. This section will detail the characteristics of these three architectures and mention the differences between them.

Sometimes you might hear, "Yeah, event-driven microservices have already been doing that since 2000 with SOA, nothing new." SOA is composed of services that choreograph to accomplish a complex workflow instead of the synchronous flow of a single application like a monolith. It also uses messaging to communicate between services. At first glance, it indeed sounds similar to event-driven architectures, but they are fundamentally different.

1.3.1 SOA

SOA typically tries to build business functionality using reusable components that communicate through a decoupled medium like a network or service bus. We will focus on SOA with ESB (enterprise service bus), which is similar to event-driven architectures. The typical organization of an SOA architecture is illustrated in Figure 1-5.

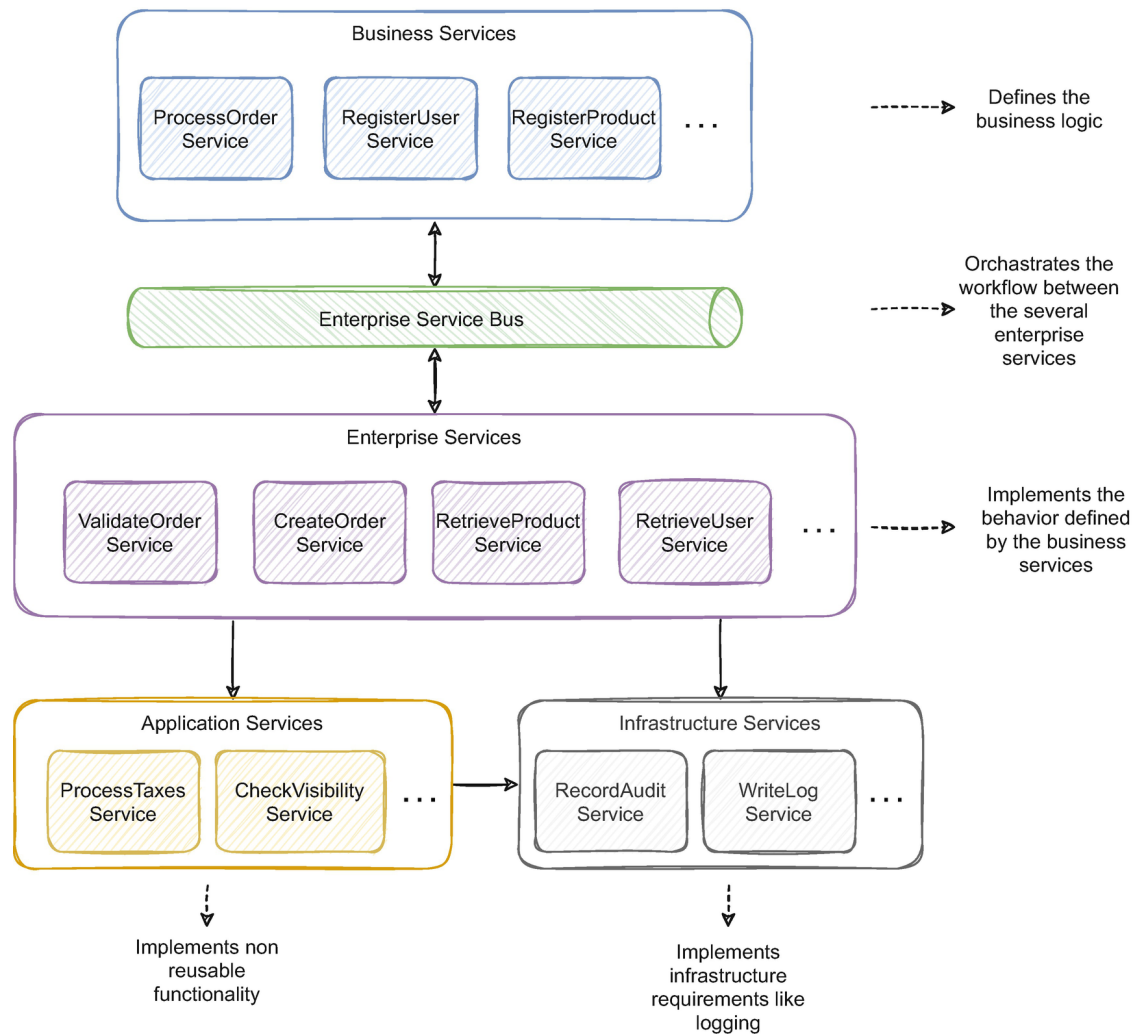


Figure 1-5 Typical service organization of SOA architectures

Illustrating with the same example of a user buying a product on an eCommerce platform, we could define a business service named “Process-Order” to handle the business logic associated with processing the order. This service determines in an abstract way what is needed for that business to process the order. The enterprise service bus orchestrates the calls needed to the enterprise services. The enterprise services implement specific behavior defined by the business services. In this case, the service bus would call each enterprise service in the correct order to process the user’s order, and the enterprise services would do the actual work and

manage the state. Enterprise services are designed to be reusable for any workflow; the application services, on the other hand, implement functionality that does not need to be reusable, like processing the order specific taxes that would be relevant just for the order workflow, for example. Infrastructure services implement cross-cutting functionality like logging or monitoring.

The mindset behind the application services is to make functionality reusable, and this is a core design concern in this kind of architecture, opposed to microservices and event-driven where sharing functionality is limited (even avoided). While SOA focuses on abstract, reusable functionality, microservice and event-driven architectures focus on organizing their components around domains.

Also, in SOA the service bus tends to become increasingly larger. Business logic tends to be added to the bus instead of the services. The bus also accounts for several responsibilities besides orchestration. The bus is usually the component that routes and knows to communicate with each service and often has logic to adapt and transform requests. The enterprise service bus tends to grow and become a monolith in its own right although the whole architecture is distributed, often having the worst of both worlds.

1.3.2 Microservice Architecture

A microservice architecture comprises several specific purpose services that interact together to accomplish a given process or workflow. The services are grouped according to a bounded context and in the scope of a given domain. They are decoupled and interact with each other using a message broker or HTTP requests. Figure [1-6](#) illustrates the same example of an eCommerce platform in a microservice architecture.

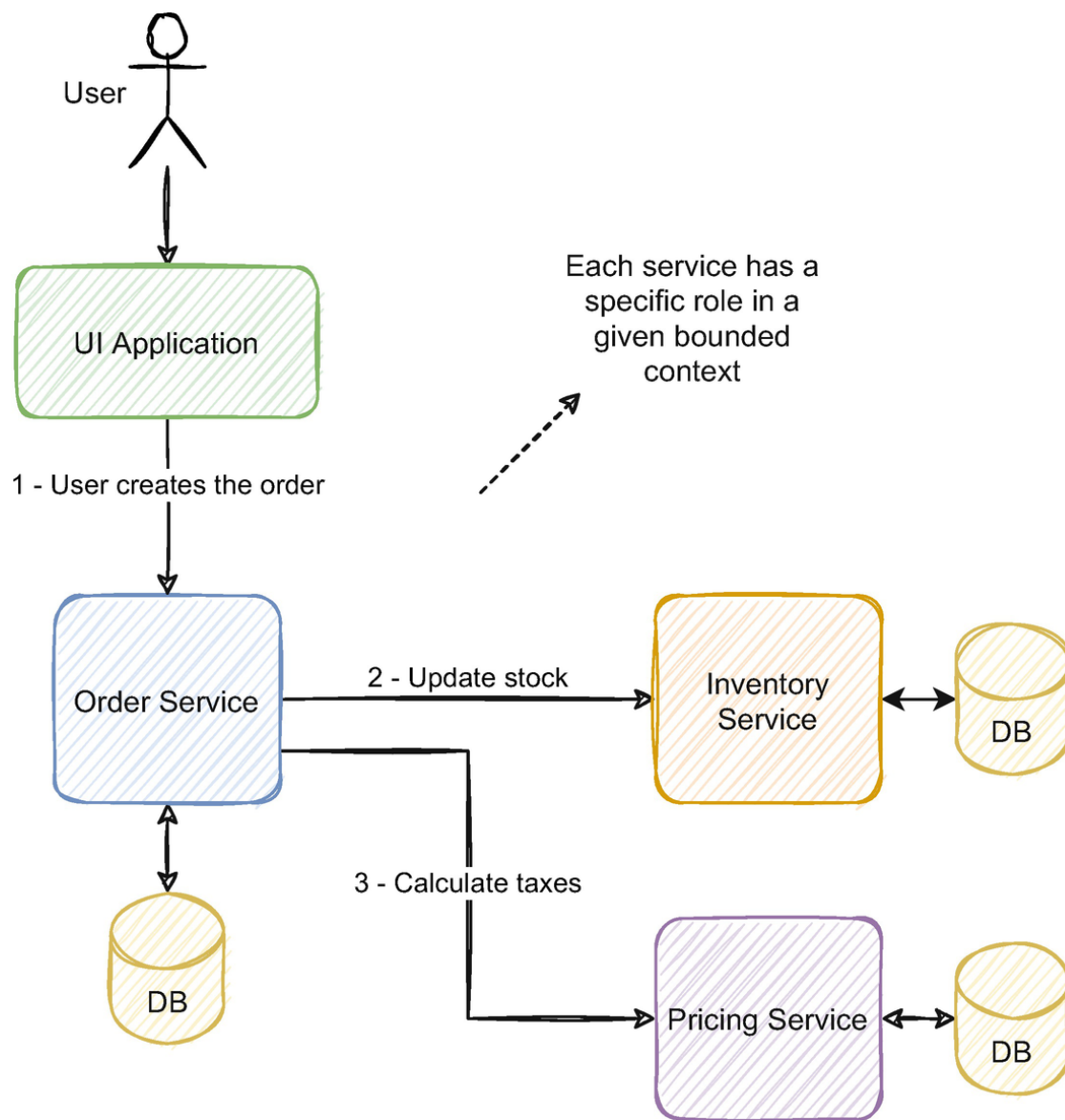


Figure 1-6 An example of the same eCommerce platform with specific microservices for each bounded context

The order service is responsible for handling the request for the new order. The service then communicates with the inventory service to manage the product's stock and the pricing service to manage the taxes. Each service is responsible for a single action and is modeled around a domain concept (order, inventory, and pricing); the workflow of processing the order is managed through the interaction between the several services. It is also important to note that each service has a database instead of a shared store; they also expose interfaces for other services to interact with them and expose only the relevant data. Opposed to SOA where reusability is a significant priority, microservices avoid sharing. Instead, they are focused on that domain and bounded context.

This kind of architecture is highly decoupled since there is a physical boundary between components. They have to communicate through the network; this poses natural bounds on each service. The organization of specific bounded contexts in the scope of a given domain allows the domain to change without affecting other unrelated components.

1.3.3 Event-Driven Microservice Architecture

As defined in the previous section, a microservice event-driven architecture shares many of the same microservice architectures' principles. But the interaction between services is focused on the usage of events. Figure 1-7 illustrates the same example of Figure 1-6 with an event-driven architecture.

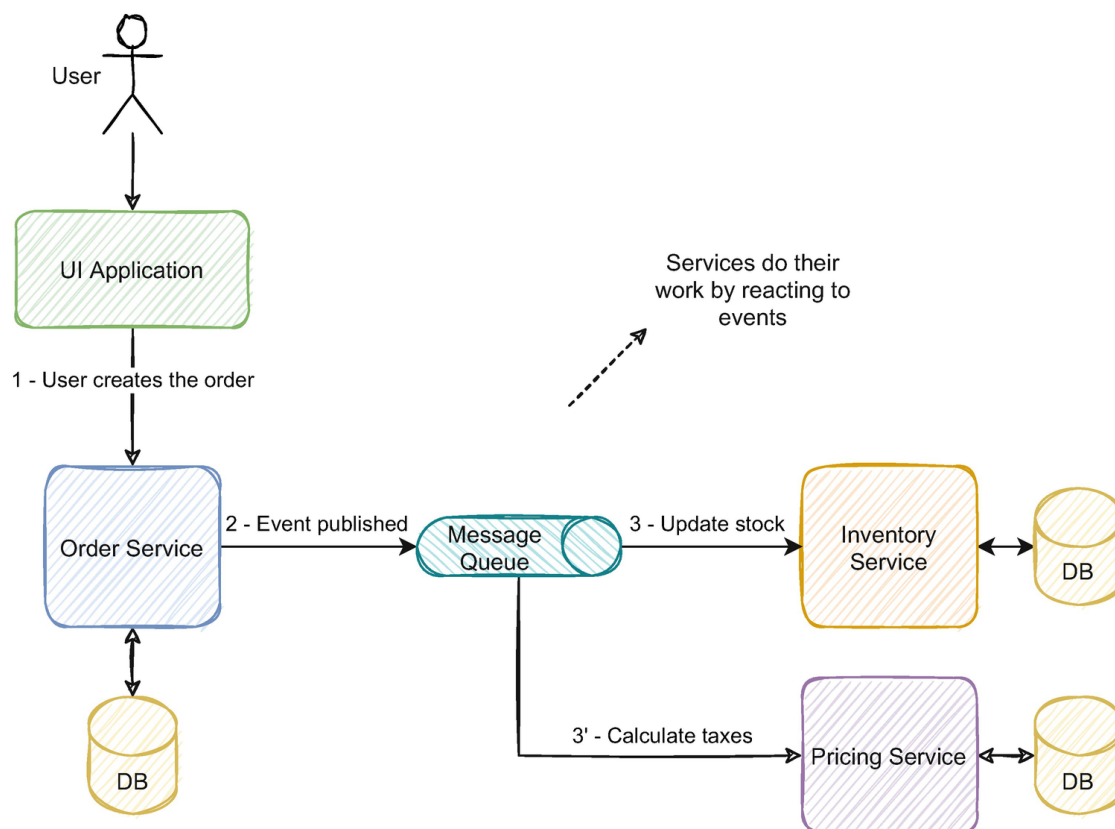


Figure 1-7 An example of the same eCommerce platform with event-driven microservices

Instead of a synchronous call between the order service and the other services, the inventory and pricing service react to the order created event from the order service. When consuming the event, they do their work asynchronously, the inventory service updates the stock, and the pricing service calculates the taxes.

By handling the communication between the services with an event broker, we further decouple the services. The order service doesn't know any of its subscribers, and the inventory and pricing service only know the event contract. It is also easy to add new consumers; we just plug them into the event queue. Imagine we had to add a subscription service that notified the users on the order's reception, we could just plug that new service into the message queue, and we don't need to change the order service. This further increases the ability to change each component and evolve their domain without affecting unrelated components or boundaries.

Architectures exclusively composed of event-driven services you will probably find in books or in proofs of concept. Most real-world architectures are a mix of asynchronous and synchronous services. Some use cases require synchronous functionality or won't simply benefit from the complexity of an asynchronous queue, and an API will suffice. The interaction between the two types of services raises challenges and needs deliberate strategies to deal with. The challenges of event-driven architectures are further detailed in [Section 1.6](#).

Event-driven architectures go beyond just reacting to events; they introduce a novel way to share their data. Typically, this was done with synchronous requests through an API, for example. APIs are useful for a real-time fetch of a small set of the latest data. Moving a large data set is difficult and can have impacts on the service and its database. We don't share databases due to the schema coupling or the impact an application can have on another. Synchronous HTTP requests can have the same effect we try to avoid when deconstructing a monolithic database.

In an event-driven architecture, it's also possible to understand the evolution of the data. Often the most critical data is the most recent data. Typical messaging produces an event and disappears once the service consumes it. However, the ability to keep the event and maintain it to be consumed again freely by the same application or future applications unlocks powerful possibilities. The event stream becomes a medium to share all relevant data occurring in real time and provides a way to understand how it changed in the past. It is also a powerful way to stream

data to other services. Each consumer can plug into the stream and create a personalized view of the state. This property enables the service to combine data from different sources and model it in the best way to its domain and querying needs.

1.4 The Promise of Event-Driven Microservices

Until now, we discussed monoliths' characteristics, the differences between the types of architectures, and what defines an event-driven architecture. This section will discuss the advantages of microservice event-driven architectures and how they can be an effective solution to many of the limits we usually find in monoliths. We will also discuss how event-driven can benefit a regular microservice architecture.

When Toyota developed lean manufacturing, the focus was on identifying and preserving value for each product; a key concept was eliminating waste and everything that didn't add value to the customer. For example, if you are buying a car, you don't care how long the car's doors travel inside the factory, although the longer the parts' path, the higher the impact on productivity and consequently on the car's cost. The factory should be composed so it minimizes the door's transport; you just care if the car has functional doors. Sometimes we find event-driven architectures compelling due to the technical challenge. Still, when a customer submits an order, it doesn't care if it is processed by a monolith or a cutting-edge architecture. The focus should be on delivering value quickly, safely at scale. How do event-driven architectures enable this?

1.4.1 Evolving Architecture

The services that compose the architecture are contained and decoupled and have a specific role in a given domain. These characteristics limit the impact of changes in the architecture; changing one service will hardly affect others. The natural segregation by bounded contexts enables the domains to change along with the business without affecting other domains. It is easy to add new services to the architecture; often the only change needed is to add the new consumer service, which doesn't require changes in the upstream services.

Having small components decoupled by message queues also enables an experimentation mindset; it is easy to try something with a new service, and if it doesn't work, decommission it. Technical debt is also naturally contained in a component or boundary due to the high decoupling between each component since each service is a different solution and an independent process. It creates the opportunity when the refactoring is too complicated and involves changing the whole service, to create a new one and descope the old one, as long as the bootstrap of a new service is straightforward.

1.4.2 Deployment

As we discussed, monoliths' deployment is risky and usually is associated with test regressions and large release cycles. Instead, event-driven microservices, due to each service's granular and decoupled nature, enable independent deployments without the need to coordinate services. It paves the way to continuous delivery; it is possible to change small increments of functionality throughout the system with little impact, quickly and sustainably. In the example depicted in Figure [1-7](#), if there were a new way to calculate the taxes in the pricing service, we would change the pricing service to calculate the new taxes and deploy only that service.

It also provides the foundation for a cultural change. When the organization has one application, where developers typically would be focused on the building stage, now it has dozens or hundreds. The focus needs to change to the whole application's life cycle; the teams need to manage developments and services from the developing phase until the deployment in production. There is a fascinating article⁶ from Microsoft which associates code ownership with more quality. This cultural change enables that ownership and autonomy.

1.4.3 Team's Autonomy

Instead of coordinating changes in a single application, teams will own and autonomously develop a set of services. Due to the services' decoupled nature, it is easy to autonomously develop those services without de-

dependencies and coordination from other teams. These properties mean decisions can be made quicker and by the people who get most impacted by them – the ones in the trenches working with the services every day and who are called to solve their incidents. It also empowers the teams to make those decisions.

Having teams owning sets of services enables functionality to be added to different domains simultaneously. Usually, the only coordination needed between teams is the contract definition; once agreed upon, both teams can develop their own services in parallel. In the example of Figure [1-7](#), if the pricing service needed to add a feature to calculate the shipping costs from the order's address, the order service might need to publish the order's address in the event and the pricing service would need to do the calculation. But once they agreed on how to reflect the address in the event, they could do their development in parallel, autonomously without dependencies from each other.

This characteristic is the building block to increase the number of teams. We often refer to technical scalability, but this allows the engineering team to be scalable. By enabling a decoupled architecture, with defined boundaries and ownership between services, we structure the way to add more teams sustainably and grow organically.

1.4.4 Flexible Technology Stack

We often hear the phrase “use the right tool for the right job.” I find it to be very overrated, there are several tools for the job, there isn't a right one, it's often about weighting the tradeoffs and preference for a set of properties above others. Despite this, some technologies are clearly more suitable for some roles than others. For example, if we need geo-distribution between several data centers without significant query capabilities, using Cassandra for that use case could be a good choice, but it would hardly be a good choice to use in all the platform services due to the query limitations. It would be hard to manage different database technologies in a single application like a monolith (although possible with a modular monolith but we don't often see that). On the other hand, on an

event-driven architecture since all components are independent and decoupled, it would be easy to adopt different technologies for each service.

Another argument we might see a few years ago would be to use a full-stack JavaScript application to facilitate development with a single programming language. Meanwhile, we learned that it's not a good approach⁷ and polyglot programming might be preferable.⁸ Having decoupled services enables a polyglot environment, which in a monolith would be very hard to do.

1.4.5 Resilience and Availability

An event-driven architecture enables higher resilience in the sense a single fault can't, or hardly will, crash the whole system. Monolithic applications are more susceptible to this since a critical fault in an infrastructure part of the application (like a memory leak) can crash the whole application, even the modules that weren't changed. In an event-driven microservice architecture, these kinds of faults are naturally bound to that component, so if one fails, the others will keep operating.

Event-driven achieves a higher resilience than synchronous microservice applications due to the decoupling provided by the message queues. In a distributed microservice application with a network of HTTP requests, we often see that they are susceptible to cascading failures. If one service fails, it will impact all the upstream services and the failure will propagate along the chain. We illustrate this situation in Figure [1-8](#). This topic is further detailed in Chapter [3](#).

As we saw in Subsection [1.1.2](#), we can scale a monolith, but it is all or nothing; we scale the whole application, not just the modules we need. If the platform received a peak in orders, we would hardly need to scale the reporting module, for example. Microservices enable to scale only the services that need scaling. This enables the system's resources to diminish or expand with the business needs. It is also cost-effective since we are mobilizing our resources to the components that need them.

Event-driven microservices are often consumers on an event broker; adding new consumers is usually straightforward. The ability to effortlessly add new consumer instances provides natural horizontal scaling without the need for load balancers. Scaling in monoliths is also usually limited by the database; by having services with independent databases, we distribute the data between the services, removing that limitation from the system.

1.4.7 The Past on Demand

As we discussed in the last section, event-driven architectures can share data through events. The sharing isn't limited to the information that is happening at the present time; instead, it provides the full history of events until now. In typical systems, to migrate data from one service to another, we require long processes that query the originating service API, usually with throughput limitations to restrict the impact on the system. It also has complex logic to deal with changes happening during the migration. For example, if we are running a process to migrate stock, a product might have two units of stock at the start of the migration. While the migration is running, a user might have bought those units. So the migration process must account for changes while the migration is occurring, which is prone to tricky corner cases.

With event-driven architectures, we handle this naturally since we start reading from the beginning of history; when we reach the end, it is guaranteed that every change that happened in between is reflected. Streaming data is more sustainable than querying; it depends solely on the service that needs the data. There is also value in the past; the business can use it to learn and infer. If a new projection of the data is need-

ed, we can use that stream to create a unique projection; the data didn't disappear and is readily available for anyone to use.

1.5 When Should You Use Event-Driven Microservices?

This section will briefly discuss what signs might be relevant when considering a move to an event-driven architecture. We will also question if it is the right decision to make. Microservice, in general, has been a fashionable technological choice and has been adopted by many companies, many times thoughtlessly, what ThoughtWorks names Microservice Envy.⁹ As we saw in the previous section, combined with event-driven, it has powerful properties. But should we blindly rush to adopt it?

When I started using C# Linq (a language component that adds native querying to .net) many years ago, I started using it for everything. Need to select the first element of an array? Linq it is. Until someone, much wiser than me, asked me if I wasn't overdoing it. Me at the time, a know-it-all youngster, baffled by the comment pretentiously recited all the benefits of Linq in every possible use case. I did mend my ways regarding Linq, but isn't that what we often do with popular technologies? Currently, microservices suffer the same thoughtless adoption. There's an interesting quote by Kelsey Hightower that says, "You haven't mastered a tool until you understand when it should not be used," which symbolizes this very well.

The first thing to ask when moving to an event-driven is why are we adopting that architecture? As we saw in Subsection [1.1.3](#), there are limits to monoliths that can draw the business growth to a halt. We look to solve many of those limits with microservices. We need to have a clear reason, which should be a top priority to solve while migrating. As engineers, we try to base our decisions on data and metrics; it would be great to say "Ok, after 30 days without releases we definitely need to move to microservices." Obviously, it doesn't work that way. However, there might be strong signs indicating we might benefit from a microservice event-driven architecture:

- The team outgrows the application: The communication and coordination overhead dramatically exceeds the development effort. Feature merges are gigantic and error-prone, with common conflicting touch-

points. These can still be managed with a modular monolith, assigning modules to different teams promoting independence and autonomy, but only goes so far. The natural bounds of microservices are often a good fit.

- **Scaling:** We can't scale anymore. This is a tricky one; when you think you can't scale, most likely you still can; probably it is just too expensive. As we mentioned, we can scale monoliths with duplicate instances of the same application; this is a valid alternative. The database, however, is typically the pain point. Usually, it can only scale vertically or with a different technology. Event-driven microservices are designed to be horizontally scalable, which helps solving this issue.
- **Deprecated technology stack:** The application technology is no longer supported or is outdated. It would be possible to upgrade the existing application incrementally, especially in a modular monolith. However, most of the time, it is challenging and expensive.
- **Embrace a continuous delivery mindset:** Although more challenging, nothing prevents a monolith from adopting a continuous delivery mindset. If the release cycles are massive, imply lengthy regressions, and if the solution is unbearably large, a possible solution could be a modular monolith. However, often transitioning to a modular monolith is hard; microservices need a foundation of automated releases and DevOps culture, which can promote the transition.

Most of these signs require a significant scale; we wouldn't say every startup needs to start with a monolith,¹⁰ but they often are good examples of how they don't benefit¹¹ from a complex microservice architecture from the ground up. A recent company is still trying to figure out what works and what doesn't; we as developers are still trying to understand what domains exist and their boundaries. Without stable boundaries between domains, we might have to redraw them later, which can be costly in existing microservice architectures.

We discussed how the decoupled nature of event-driven architectures and the localized scope of microservices can facilitate swapping components. This characteristic can seem antagonistic to the associated cost of redefining boundaries. Large restructures of domain boundaries usually impact several services from different boundaries. They often require the

domain to change, and that usually impacts changing the domain logic and models, database schemas, and event contracts of several services, which often impact several teams and have complex dependencies. We should avoid this kind of profound changes to the architecture due to the sheer effort they imply. Often it is more beneficial to have a single broad application where we can understand what boundaries exist without hampering time to market. When we have a sound understanding of the domains, we can start migrating to a distributed architecture.

While a single application is easy to monitor and observe logs, event-driven microservices require a solid foundation of release maturity through pipeline automation, centralized logging and monitoring, automated testing, and message brokers. All of these have an associated cost to bootstrap and maintain that the company needs to account for. It definitely pays up in complex large businesses but is debatable if a small business would benefit largely from it.

A large number of components in microservice architectures also benefit autonomy and ownership, but centralized decision becomes hard. Often, SOA architectures with ESB had a centralized team that decided the message contracts, for example. With event-driven microservices, that management becomes problematic and a bottleneck; although standard guidelines are useful, we should delegate the autonomy of the decision to the teams, which might not be the best situation for a company that wants to retain central control.

1.6 Overview of the Challenges in Event-Driven Architectures

As we discussed in the last sections, a microservice event-driven architecture can solve complex problems and be the foundation of a truly scalable, resilient, and evolutionary architecture. Adopting one also means we are opening a Pandora's box of new kinds of challenges. Many of the characteristics we always took for granted in a monolith suddenly disappear. Distributed systems also carry a new tier of challenges of their own and require deliberate strategies to deal with. This section will briefly discuss those challenges.

A fundamental step to designing an event-driven architecture is getting the size and scope of the services right. There is no formula for this and it can become subjective. The most useful strategy is to follow DDD (domain-driven design) and get the bounded contexts between the services right. This, however, requires a strong knowledge of the system and its domains. As we saw in the previous sections, changing these boundaries later on might be costly and time-consuming. We detail the service design and DDD in Chapter 3. Chapter 4 discusses patterns to organize microservices.

In a monolith, we are used to enjoying strong consistency; any change is seen anywhere in the application instantaneously. The presence of event brokers between the services turns each service asynchronous. The asynchronous nature of the services produces what is known as eventual consistency; given enough time, all changes will eventually propagate throughout the system. However, while the changes are still propagating, different system parts might return old data to customers and other services inside the architecture. This raises the challenge of how to deal with stale data. Chapter 5 further details and proposes solutions to handle eventual consistency.

Eventual consistency and the asynchronous nature of the services require a different approach to UI design. Typical UIs induce the user into false strong consistency, as updates might take some time to occur. Messages like “updates might take some time to be reflected” on the UI are common and also a dubious user experience. Chapter 9 further details this topic.

Dealing with concurrency in a single application is also straightforward; we can use traditional ways to deal with concurrent threads like locking or mutexes. In an event-driven architecture, however, we scale by having multiple instances of the same service. In-memory concurrency mechanisms no longer work since concurrent requests or events can target different instances. Message ordering can also be an issue if the order of events is switched, if the service ends up processing an old event after the most recent one can become inconsistent. Although there are brokers that guarantee the order, most don’t, and retrying can disorder messages.

Chapter [7](#) details how to achieve better consistency with idempotent messages. Chapter [6](#) details strategies to deal with concurrency and unordered messages.

Event-driven architectures enjoy a high level of decoupling. The domain flow itself couples them as business processes progress through the boundaries. Often, the only sharing point between the services is the message's contracts. Contracts become pivotal to the system design as the event schema and its size often is a topic of lively discussion. Events can be small or large; depending on how we design them, both approaches have tradeoffs. However, they heavily impact the systems that need them and the overall architecture, as we detail in Chapter [8](#).

Support teams often need to do quick data fixes on applications. Monolithic applications support this since a monolithic database typically backs them up. When we move to a distributed architecture with dozens or hundreds of services, these quick fixes aren't so straightforward. In an event-driven architecture, correcting something often means sending a command with the opposite operation (if we add stock using an `AddStock` command incorrectly due to a bug, we need to do the reverse operation and send a `RemoveStock` command, for example). Usually, to do so requires custom tooling from the ground up to allow these kinds of operations.

As we discussed before, in an event-driven architecture, the message brokers allow the components to be highly decoupled. The decoupling enables the architecture to be highly evolutionary and the components to come and go as they wish. It is a great property to allow the system to evolve along with the business. However, when we have many decoupled components, it gets hard to understand the flow between the services. Testing the end-to-end flow, it's impractical due to the interaction of several components. Each component is also independently deployable, raising the challenge of guaranteeing the quality of the end-to-end flow every time we deploy a single component. Chapter [10](#) further details the challenges and strategies of quality assurance.

Different teams are responsible for different sets of services. Adding a business feature might impact several teams, as features often involve

several domains. Although the decoupling between the services promotes parallel working, sometimes complex dependencies between several teams arise. Chapter [10](#) further details the organizational impact of event-driven architectures.

When technologies become popular, people tend to see them as a magic solution to most of our problems. We sometimes rush to turn a blind eye to their issues and focus on the benefits they can bring. However, it's always about weighing the tradeoffs. There are types of challenges that we can only feel their true extent in production. Certainly, proofs of concept are great to test a specific objective, but most of the time, we need to run a technology at scale in production to really learn them and understand the reach of their limitations. The added complexity, the distributed characteristics, and the asynchronous nature of event-driven architectures pave the way to complex challenges. To manage them, we require deliberate strategies. This book approaches those challenges and proposes strategies to deal with them.

1.7 Summary

- There are different types of monoliths; the hardest to deal with is patchwork monoliths due to entwined logic and lack of boundaries.
- Modular monoliths can be an adequate architecture choice and offer several advantages and reduced complexity over distributed architectures.
- Businesses can outgrow monoliths, and given enough time, monoliths' limits can constrict business growth.
- Event-driven architectures can facilitate the migration from a monolith to a microservice architecture by promoting high decoupling between components.
- Event-driven architectures accomplish higher-level processes through the flow of events of several small, specific purpose services.
- An event-driven architecture facilitates a medium to share past and present data in real time to any service. By easing the decoupled sharing of every data that occurred and is occurring in the system unlocks powerful possibilities.

- An event-driven architecture's components enjoy a set of properties that are the foundation for a scalable, evolutionary, and decoupled architecture.
- There are situations where event-driven architectures aren't the best option. We also need an exact reason when deciding to move to a microservice event-driven architecture; that reason should be a top concern while migrating.
- The distributed, complex, and asynchronous nature of event-driven architectures has difficult challenges we must address from the ground up.

Footnotes

¹ A common anti-pattern in software architecture,

https://en.wikipedia.org/wiki/Big_ball_of_mud

² See Dan Manges, "The Modular Monolith: Rails Architecture," Medium, January

23, 2018, https://medium.com/@dan_manges/the-modular-monolith-rails-architecture-fb1023826fc4

³ See "Cookie Cutter Scaling," November 29, 2011,

<https://paulhammant.com/2011/11/29/cookie-cutter-scaling/>

⁴ See Kirsten Westeinde, "Deconstructing the Monolith: Designing Software that Maximizes Developer Productivity," February 21, 2019, <https://shopify.engineering/deconstructing-monolith-designing-software-maximizes-developer-productivity>

⁵ See Neal Ford, Rebecca Parsons, and Patrick Kua, "Building Evolutionary Architectures," www.thoughtworks.com/books/building-evolutionary-architectures

⁶ See "Don't Touch My Code! Examining the Effects of Ownership on Software Quality," September 5, 2011, www.microsoft.com/en-us/research/wp-content/uploads/2016/02/bird2011dtm.pdf

⁷ See ThoughtWorks Technology Radar, "Node overload," May 19, 2020, www.thoughtworks.com/radar/platforms/node-overload

[8](#) See ThoughtWorks Technology Radar, “Polyglot programming,” April 29, 2020,
www.thoughtworks.com/radar/techniques/polyglot-programming.

[9](#) See ThoughtWorks Technology Radar, “Microservice envy,” November 14, 2018,
www.thoughtworks.com/radar/techniques/microservice-envy

[10](#) Further detailed by Stefan Tilkov, “Don’t start with a monolith,” June 9, 2015,
<https://martinfowler.com/articles/dont-start-monolith.html>

[11](#) Further detailed by Martin Fowler, “MonolithFirst,” June 3, 2015,
<https://martinfowler.com/bliki/MonolithFirst.html>

[Support](#) [Sign Out](#)

©2022 O'REILLY MEDIA, INC. [TERMS OF SERVICE](#) [PRIVACY POLICY](#)